Caso: Classificador Naïve Bayes na Ferramenta scikit-learn

A04 – Introdução ao Processamento de Linguagem Natural (PLN)

Eduardo Corrêa Gonçalves

12/03/2026

Sumário

Introdução

O que é scikit-learn ?

Treinamento do classificador: passo a passo

Definição da base de dados de treinamento

Separação das partes X (features) e Y (classe)

Pré-processamento da parte X

Treinamento do modelo

Entendendo o modelo gerado

Classificação de novos textos: passo a passo

Importação da base de dados com os novos documentos

Pré-processamento

Classificação

Introdução (1/1)

- **O que é scikit-learn ?**
  - A scikit-learn (https://scikit-learn.org/) é uma popular ferramenta livre de **aprendizado de máquina tradicional*** (AM) desenvolvida em Python.
  - Oferece implementações dos principais algoritmos de AM:
    - naïve bayes;
    - random forest;
    - modelos lineares;
    - rede neural *feed forward*;
    - clustering (k-médias, hierárquico, densidade)
    - funcionalidades para pré-processamento de dados.
    - *outras...*
  - Também é conhecida como **sklearn**.

** Não é voltada para redes neurais profundas.*

Introdução (2/2)

- **O que é scikit-learn ?**
  - A ferramenta é bem abrangente! Oferece muitas coisas!
  - Neste curso, vamos apresentar algumas de suas funcionalidades de maneira gradual, especialmente nas aulas com os estudos de caso.
  - Começaremos hoje examinando os passos básicos para treinar e utilizar um modelo Naïve Bayes Multinomial (NB) para classificação de textos.

Link para notebook com os exemplos da aula:

- O código apresentado nos slides a seguir está disponível no seguinte notebook:

- https://colab.research.google.com/drive/1Dl-Z2L2zO82a4bmMlQwqUSotq3qJeKe9?usp=sharing

Treinamento do Classificador – passo a passo (1/12)

- **Os <u>4 passos</u> para <u>treinar</u> um classificador na scikit-learn**
  1. Importação da base de dados de treinamento.
  2. Separação das partes X (*features*) e Y (classe).
  3. Pré-processamento da parte X.
  4. Treinamento do modelo.

- **Não** é um **esquema fixo**! Dependendo do problema, você pode alterar esse fluxo. Exemplos:
  - Muitas vezes, a etapa 3 precisará ser dividida em várias subetapas (quando é necessário aplicar várias técnicas de pré-processamento) .
    - Outras vezes, ela pode nem ser necessária.
  - Algumas pessoas invertem a ordem de execução das etapas 2 e 3.
  - Outras separam X e Y (etapa 2) apenas no momento do treinamento (etapa 4)

Treinamento do Classificador – passo a passo (2/12)

- **Os <u>4 passos</u> para <u>treinar</u> um classificador na scikit-learn**
  1. Importação da base de dados de treinamento.
  2. Separação das partes X (*features*) e Y (classe).
  3. Pré-processamento da parte X.
  4. Treinamento do modelo.

- Vamos demonstrar cada etapa utilizando o exemplo da aula passada (Jurafsky e Martin, 2026):
  - Modelo treinado com base de dados de opiniões sobre filmes. Cada opinião é classificada como positiva (+) ou negativa (-).

comentário (documento)	**sentimento (classe)**
just plain boring	-
entirely predictable and lacks energy	-
no surprises and very few laughs	-
very powerful	+
the most fun film of the summer	+

Treinamento do Classificador – passo a passo (3/12)

- **PASSO 1: importação da base (BD) de treinamento**
  - No cenário mais típico, a base de treino estará estruturada em um **arquivo CSV**.
    - O arquivo CSV será importado para um **DataFrame** pandas.
    - Porém, no exemplo de hoje criaremos o DataFrame na "mão", já que o BD de nosso exemplo é muito pequena.

```
import pandas as pd
# seguindo a convenção, utilizaremos:
# . 0 p/ representar a classe dos textos de opinião negativa
# . 1 p/ representar a classe dos textos de opinião positiva

D = pd.DataFrame({"comentario": ["just plain boring",
                                 "entirely predictable and lacks energy",
                                 "no surprises and very few laughs",
                                 "very powerful",
                                 "the most fun film of the summer"],
                  "sentimento": [0, 0, 0, 1, 1]})

D # apenas para visualizar o DataFrame no Colab
```

Treinamento do Classificador – passo a passo (4/12)

- **Resultado ao final do PASSO 1**
  - BD de treinamento estará estruturado em um DataFrame

D

	comentario	sentimento
0	just plain boring	0
1	entirely predictable and lacks energy	0
2	no surprises and very few laughs	0
3	very powerful	1
4	the most fun film of the summer	1

Treinamento do Classificador – passo a passo (5/12)

- **<u>PASSO 2</u>: separação das partes X (*features*) e Y (classe)**
  - No momento de treinar o modelo a sklearn exige que seja indicado quem é o conjunto de *features* X e quem é a classe Y.
    - Então, para facilitar, podemos fazer logo a separação (embora não seja algo obrigatório).
    - Veja que chamei X de "X_raw"*... (literalmente "X cru").
    - É porque na próxima etapa ele será pré-processado e só depois disso vai virar o "X_final", no formato **bag of words**, que o NB requer para treinar o modelo.

```
# X_raw contém o texto (col; 0) em um DataFrame
# IMPORTANTE: - é preciso usar [] em volta da coluna desejada
# p/ forçar a geração de um DataFrame! Se isso não for feito,
# como o fatiamento envolve apenas uma coluna, seria gerada uma
# Series (espécie de array)
# A sklearn exige que a parte X seja uma matriz
# ou DataFrame, mesmo que possua uma coluna só
X_raw = D.iloc[:, [0]]

# Y é um array (na verdade uma Series) com o sentimento (col 1)
# IMPORTANTE: já a parte Y tem que ser um array ou Series !!!
Y = D.iloc[:, 1]
```

** "raw text" (texto cru) é um jargão da PLN para referenciar o texto original de um corpus (dataset)*

Treinamento do Classificador – passo a passo (6/12)

- **Resultado ao final do PASSO 2**
  - X_raw (textos originais dos comentários) e Y (classe de cada texto) terão sido separados.
  - X ainda estará no formato original e precisará ser transformado em uma *bag of words* antes do treinamento do modelo de classificação.

```
print('- X (original):'); print(X_raw)
print('- Y:'); print(Y)
```

```
- X (original):
                                comentario
0                       just plain boring
1  entirely predictable and lacks energy
2        no surprises and very few laughs
3                           very powerful
4         the most fun film of the summer
```

```
- Y:
0    0
1    0
2    0
3    1
4    1
Name: sentimento, dtype: int64
```

IBGE
Escola Nacional de Ciências Estatísticas ENCE

Treinamento do Classificador – passo a passo (7/12)

- **PASSO 3: Pré-processamento da parte X**
  - Conversão cada texto do DataFrame "X_raw" p/ Bag of Words (BoW).
  - Gera as bag of words usando a classe `CountVectorizer` do módulo `sklearn.feature_extraction.text`.
    - CountVectorizer é um módulo de pré-processamento oferecido pela sklarn...
    - ... que serve exatamente para gerar uma matriz do tipo bag of words...
    - ... a partir de uma coluna de um DataFrame contendo textos.
      - Cada texto vira uma linha da matriz e cada coluna é uma palavra do vocabulário do corpus.

Treinamento do Classificador – passo a passo (8/12)

- **PASSO 3: Pré-processamento da parte X**
  - Abaixo o código comentado...

```
from sklearn.feature_extraction.text import CountVectorizer

#-----------------------------------------------------------
# 1 - instancia um objeto da classe CountVectorizer
#-----------------------------------------------------------
c_vect = CountVectorizer()

#-----------------------------------------------------------
# 2- gera matriz c/ contagem de frequência dos tokens
#          (isto é, as bag of words de cada texto)
#-----------------------------------------------------------
bow = c_vect.fit_transform(X_raw['comentario']).toarray()

#-----------------------------------------------------------
# 3- para ficar "mais bonito", podemos transformar a
# matriz com as bag of words (bow) em um DataFrame chamado X,
# jogando os nomes dos tokenscomo nomes das colunas.
#
#          Assim o modelo guardará os nomes das features
#          (como veremos em breve)
#-----------------------------------------------------------
tokens = c_vect.get_feature_names_out()  # lista de tokens
X = pd.DataFrame(bow, columns=tokens)
```

IBGE | Escola Nacional de Ciências Estatísticas ENCE

Treinamento do Classificador – passo a passo (9/12)

- **Resultado ao final do PASSO 3**
  - Teremos um DataFrame X (nosso X final) contendo os textos originais convertidos para formato BoW, que é o formato exigido pelo NB !

```
# vamos imprimir tudo que foi gerado, só como curiosidade
print(tokens)          # tokens
print(bow)             # matriz c/ as bag of words
X                      # DataFrame c/ as bag of words
```

```
['and' 'boring' 'energy' 'entirely' 'few' 'film' 'fun' 'just' 'lacks'
 'laughs' 'most' 'no' 'of' 'plain' 'powerful' 'predictable' 'summer'
 'surprises' 'the' 'very']
```

Tokens do vocabulário do corpus

```
[[0 1 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0]
 [1 0 1 1 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0]
 [1 0 0 0 1 0 0 0 0 1 0 1 0 0 0 0 0 1 0 1]
 [0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1]
 [0 0 0 0 0 1 1 0 0 0 1 0 1 0 0 0 1 0 2 0]]
```

Matriz com cada BoW (col0='and', col1='boring' ... col19='very').

DataFrame com cada BoW (mais fácil para trabalhar do que a matriz)

	and	boring	energy	entirely	few	film	fun	just	lacks	laughs	most	no	of	plain	powerful	predictable	summer	surprises	the	very
0	0	1	0	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0
1	1	0	1	1	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	0
2	1	0	0	0	1	0	0	0	0	1	0	1	0	0	0	0	0	1	0	1
3	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1
4	0	0	0	0	0	1	1	0	0	0	1	0	1	0	0	0	1	0	2	0

Treinamento do Classificador – passo a passo (10/12)

- **PASSO 3: Pré-processamento da parte X**
  - Algumas observações sobre a tokenização (separação das palavras) padrão da classe `CountVectorizer` :
    - **Converte** textos para **minúsculo** (lowercase=True)
    - Considera como tokens as sequências com 2 ou mais caracteres formadas por **letras**, **números** ou **underscore** (token_pattern='(?u)\b\w\w+\b'). Ou seja:
      - tokens com **um único caractere** são **descartados** (ex.: "0", "a")
      - sinais de **pontuação** são **descartados**
    - É **possível** especificar uma **lista de stop words** (palavras irrelevantes, como preposições e artigos) utilizando o parâmetro stop_words (não foi feito no exemplo).

Treinamento do Classificador – passo a passo (11/12)

- **PASSO 4: Treinamento do classificador**
  - (1). **importar o pacote** com o algoritmo que você quer usar, nesse caso o Naïve Bayes Multinominal (pacote `MultinomialNB` da scikit)
  - (2). **Instanciar o classificador**, indicando os parâmetros que você deseja *("instanciar" é como carregar o algoritmo em memória para posterior utilização*).
  - (3). **Usar o método** (função) **fit**, para fazer o treinamento do classificador, indicando o X e o Y. Nesse caso:
    - X é o DataFrame com as BoWs
    - Y é o vetor com as classes, que já separamos lá no Passo 2.

```
#1. importa o pacote com a classe MultinomialNB
from sklearn.naive_bayes import MultinomialNB

#2. instancia um objeto da classe MultinomialNB
modelo = MultinomialNB()

#3. utiliza o método "fit" para treinar o modelo
modelo.fit(X, Y)
```

Treinamento do Classificador – passo a passo (12/12)

- **Resultado ao final do PASSO 4**
  - **Classificador NB está construído!!!** Agora podemos utilizá-lo.
  - Abaixo um código onde imprimimos o modelo e suas propriedades (*assim podemos **entender o modelo gerado** e **bater com o exemplo** da aula teórica sobre o NB*)

```
print(modelo.feature_names_in_) # nomes das features
```

```
['and' 'boring' 'energy' 'entirely' 'few' 'film' 'fun' 'just' 'lacks'
 'laughs' 'most' 'no' 'of' 'plain' 'powerful' 'predictable' 'summer'
 'surprises' 'the' 'very']
```

```
print(modelo.n_features_in_) # total de features
```

```
20
```

```
print(modelo.class_count_) # contagem das classes
                           # 3 negativas (classe 0) 2 positivas (classe 1)
```

```
[3. 2.]
```

```
print(modelo.feature_count_)     # contagem da freqs de cada
                                 # feature em cada uma das 2 classes
```

```
[[2. 1. 1. 1. 1. 0. 0. 1. 1. 1. 0. 1. 0. 1. 0. 1. 0. 1. 0. 1.]
 [0. 0. 0. 0. 0. 1. 1. 0. 0. 0. 1. 0. 1. 0. 1. 0. 1. 0. 2. 1.]]
```

Classificação de novos textos – passo a passo (1/7)

- **Os <u>3 passos</u> para <u>utilizar</u> um classificador treinado na scikit-learn**
    1. Definição ou importação dos novos documentos (textos) – *ou seja os documentos a serem classificados pelo modelo*
    2. Pré-processamento dos novos documentos
    3. Utilização do modelo treinado para classifica-los

- Mais uma vez, **não** é um **esquema fixo**!
- Dependendo do problema ou da sua preferência, você pode alterar esse fluxo.

Classificação de novos textos – passo a passo (2/7)

- **<u>Passo 1:</u> Definição ou importação dos novos documentos**
  - Em nosso exemplo, defini manualmente um documento novo em um DataFrame (o mesmo usado na aula teórica sobre o NB)
    - *"predictable with no fun"*
  - Mas também é possível importar um dataset com vários documentos novos (na sklearn, posso aplicar o modelo para classificar vários textos de uma vez)

```
novoDoc = pd.DataFrame({"comentario": ["predictable with
no fun"]})
print('- Novo documento:'); print(novoDoc)
```

```
- Novo documento:
                  comentario
0  predictable with no fun
```

Classificação de novos textos – passo a passo (3/7)

- **Passo 2: Pré-processamento dos novos documentos**
  - Os novos documentos precisarão ser transformados em uma BoW.
  - E essa **BoW** precisará ser **compatível** com a que foi criada para o **treinamento** do modelo.
    - Isso é: precisa ter o **mesmo tamanho** e os **mesmos tokens** na mesma ordem
    - **Tokens desconhecidos** devem ser **descartados**.
  - Felizmente isso é muito simples na sklearn, ela já está preparada para essas situações.
    - Basta usar o mesmo objeto `CountVectorizer` usado para converter os textos de treino (que chamamos de c_vect no código).
    - A diferença é que agora devemos chamar o método transform (e **não** o **fit_transfom**).

Classificação de novos textos – passo a passo (4/7)

- **<u>Passo 2:</u> Pré-processamento dos novos documentos**
  - Abaixo o código e o resultado gerado.
  - A palavra "with" é descartada, pois ela não existia no vocabulário de treino.

```
# ---------------------------------------------------------
# PASSO 2. transforma o novo documento com o mesmo
# tokenizador usado para treinar o classificador. Assim a #
palavra desconhecida "with" vai ser jogada fora
# ---------------------------------------------------------
novoDoc = c_vect.transform(novoDoc['comentario']).toarray()
novoDoc = pd.DataFrame(novoDoc, columns=tokens)

print('- Bag of words do novo documento:'); print(novoDoc)
```

```
- Bag of words do novo documento:
   and  boring  energy  entirely  few  film  fun  just  lacks  laughs  most  \
0    0       0       0         0    0     0    1     0      0       0     0

   no  of  plain  powerful  predictable  summer  surprises  the  very
0   1   0      0         0            1       0          0    0     0
```

Classificação de novos textos – passo a passo (5/7)

- **Passo 3: Classificação dos novos documentos**
  - Consiste simplesmente em **aplicar o modelo** treinado (em nosso caso um NB Multinominal) para **classificar o novo documento**.
  - Isso é feito com o método predict do modelo.
    - Ele vai retornar uma das classes possíveis: “0” (negativo) ou “1” (positivo).
  - **IMPORTANTE**: podemos fazer o predict de vários documentos novos de uma vez... Nesse caso, seria retornada uma lista contendo a classe predita para cada um deles.

```
# ------------------------------------------------------------
# PASSO 3. utiliza o modelo MultinomialNB treinado para
# classificar o novo documento
# ------------------------------------------------------------
classe_predita = modelo.predict(novoDoc)
```

Classificação de novos textos – passo a passo (6/7)

- **Resultado ao final do Passo 3**
  - Novo(s) documento(s) classificado(s) pelo modelo !!!
  - Veja no código abaixo a impressão do resultado:

```
print('- A classe do novo documento é: ', classe_predita)

print("(opinião negativa)" if classe_predita == 0 else
"(opinião positiva)")
```

```
- A classe do novo documento é:  [0]
(opinião negativa)
```

Classificação de novos textos – passo a passo (7/7)

- **Também é possível imprimir a <u>probabilidade de cada classe</u>**
  - Mas não esqueça que é a probabilidade é computada aplicando as suposições ingênuas do NB !!! (*que aliás, é a prob usada pelo modelo para definir a classe do objeto*)

$$P(c \mid d) = P(c) \times P(f_1 \mid c) \times P(f_2 \mid c) \times P(f_3 \mid c) \times \ldots \times P(f_n \mid c)$$

```
print('- As probabilidades para classe 0 ("opinião negativa")
e classe 1 ("opinião positiva") são: ')

print(modelo.predict_proba(novoDoc))
```

```
- As probabilidades para classe 0 ("opinião negativa") e classe 1 ("opinião positiva") são:
[[0.65054103 0.34945897]]
```

Referências

- JURAFSKY, D.; MARTIN, J. H. Naive Bayes, Text Classification, and Sentiment. *In*: JURAFSKY, D.; MARTIN, J. H. **Speech and Language Processing**. 3. ed. (Jan. 6, 2026 draft). [*s.l.*] : [*s.n.*], 2026. Appendix B, p. 1-23. Disponível em: https://web.stanford.edu/~jurafsky/slp3/. Acesso em: 24 fev. 2026.

- scikit-learn - Naive Bayes. Disponível em: https://scikit-learn.org/stable/modules/naive_bayes.html. Acesso em: 09 mar. 2026

- scikit-learn - MultinomialNB. Disponível em: https://scikit-learn.org/stable/modules/generated/sklearn.naive_bayes.MultinomialNB.html. Acesso em: 09 mar. 2026